ADAR'S PANDAS NINJA GUIDE

PANDAS

COMPLETE MASTERY HANDBOOK

PART TWO

Text Parsing · Feature Engineering · Auditing · Sorting · Regional Filters

	country	region	population	leader
0	Norway	N. Europe	5.4M	—
1	Nigeria	W. Africa	206M	Buhari
2	India	S. Asia	1.38B	Modi

pd.read_csv(r”Pandas_Anime.csv”)

pd.read_csv(r'Countries.csv')

Compiled & Structured for Exam-Ready Mastery

Adar Dey · Algorithm Learner and AI Enthusiastic

PARTS 07 – 08 ✦ THEME: CHARCOAL & TEAL

Table of Contents

PART 07

Advanced Text Parsing & Feature Ingestion

Raw File Loading, Custom Parsers & Regex-Based Extraction

5 Core Topics · Fully Worked Examples & Outputs

PART 7 • TOPIC 1

Production File Ingestion & Label-Based Lookup

Raw local files are ingested using pd.read_csv(). Wrapping the file path with a raw string literal r'...' safely bypasses operating-system escape-character conflicts on Windows-style paths.

› _ PYTHON

```
# Ingesting a raw local unstructured tabular file using a raw string path
df_anime = pd.read_csv(r'Pandas_Anime.csv')

df_anime.head(5)                # top 5 records grid preview

# Verifying an individual element using label-based lookup (.loc)
df_anime.loc[3]['Title']
```

→ OUTPUT

	Rank	Title	Score
0	1	Fullmetal Alchemist: BrotherhoodTV (64 eps)...	9.10
1	2	Steins;GateTV (24 eps)Apr 2011 - Sep 2011…	9.07
2	3	Bleach: Sennen Kessen-henTV (13 eps)Oct 2022...	9.06
3	4	Gintama°TV (51 eps)Apr 2015 - Mar 2016...	9.06
4	5	Shingeki no Kyojin Season 3 Part 2TV (10 eps)…	9.05

Row 3 Title →
'Gintama°TV (51 eps)Apr 2015 - Mar 2016605,113 members'

✦ KEY INSIGHT

Real scraped data almost never arrives pre-cleaned — the Title column here bundles the show name, episode count, air dates, and member count into a single unstructured text blob.

PART 7 • TOPIC 2

Custom Text Parsing Loops — Episode Count Extraction

When no built-in method fits, a hand-written character-scanning function paired with .apply() can isolate exactly the substring nested inside a specific delimiter pair — here, standard parentheses.

› _ PYTHON

```
def extract_episode_metadata(target_text_string):
    capture_active = False
    extracted_buffer = ""
    for character in target_text_string:
        if character == ')':
            capture_active = False
        if capture_active == True:
            extracted_buffer = extracted_buffer + character
        if character == '(':
            capture_active = True
    return extracted_buffer
```

```
# Mapping the custom parser across the 'Title' column via .apply()
df_anime['Episodes'] = df_anime['Title'].apply(extract_episode_metadata)

df_anime[['Title', ‘Episodes’]] .head(5)                # top 5 records grid preview
```

→ OUTPUT

	Title	Episodes
0	Fullmetal Alchemist: BrotherhoodTV (64 eps)...	64 eps
1	Steins;GateTV (24 eps)Apr 2011 - Sep 2011…	24 eps
2	Bleach: Sennen Kessen-henTV (13 eps)Oct 2022...	13 eps
3	Gintama°TV (51 eps)Apr 2015 - Mar 2016...	51 eps
4	Shingeki no Kyojin Season 3 Part 2TV (10 eps)…	10 eps

✦ KEY INSIGHT

A boolean 'capture_active' flag toggles ON at '(' and OFF at ')', so only characters seen while the flag is True get appended to the buffer.

PART 7 • TOPIC 3

Advanced Regex Pattern Extraction — Date Range Segmentation

For more irregular patterns, Python's re module locates structured date sequences directly, with a fallback pattern for single-date entries like movie premieres.

❯_ PYTHON

```
import re
def extract_time_metadata(target_text_string):
    parenthesis_close_index = target_text_string.find(')')
    member_keyword_index = target_text_string.find('members')
    if parenthesis_close_index == -1 or member_keyword_index == -1:
        return np.nan

    raw_date_segment = target_text_string[parenthesis_close_index + 1 : member_keyword_index]

    # Primary pattern: full ranges like 'Apr 2009 - Jul 2010'
    date_match = re.search(r'[A-Za-z]{3}\s+\d{4}\s*-\s*[A-Za-z]{3}\s+\d{4}', raw_date_segment)
    if date_match:
        return date_match.group(0)

    # Fallback pattern: single premiere dates like 'Jan 2021'
    movie_match = re.search(r'[A-Za-z]{3}\s+\d{4}', raw_date_segment)
    if movie_match:
        return movie_match.group(0)

    return raw_date_segment.strip()

df_anime['Time_Range'] = df_anime['Title'].apply(extract_time_metadata)
df_anime[['Title', 'Episodes', 'Time_Range']].head(5)
```

→ OUTPUT

	Title	Episodes	Time_Range
0	Fullmetal Alchemist: BrotherhoodTV (64 eps)Apr...	64	Apr 2009 - Jul 2010
1	Steins;GateTV (24 eps)Apr 2011 - Sep 20112,473...	24	Apr 2011 - Sep 2011
2	Bleach: Sennen Kessen-henTV (13 eps)Oct 2022 -...	13	Oct 2022 - Dec 2022
3	Gintama°TV (51 eps)Apr 2015 - Mar 2016605,113 ...	51	Apr 2015 - Mar 2016
4	Shingeki no Kyojin Season 3 Part 2TV (10 eps)A...	10	Apr 2019 - Jul 2019

✦ KEY INSIGHT

Layering a primary regex pattern with a simpler fallback pattern is a robust strategy for messy real-world text that doesn't always follow one consistent format.

PART 7 • TOPIC 4

String Sanitization & Explicit Type Casting

.str.replace() performs vectorized character removal across an entire text column, and .astype() then promotes the cleaned string labels into a pure numeric dtype for downstream math.

› _ PYTHON

```
# Removing the trailing ' eps' label via vectorized string replacement
df_anime['Episodes'] = df_anime['Episodes'].str.replace(" eps", "")

# Casting the cleaned string values into a true integer dtype
df_anime['Episodes'] = df_anime['Episodes'].astype(int)

df_anime[['Title', ‘Episodes’]] .head(5)                # top 5 records grid preview
df_anime[['Title', 'Episodes']].dtypes
```

→ OUTPUT

	Title	Episodes
0	Fullmetal Alchemist: BrotherhoodTV (64 eps)...	64
1	Steins;GateTV (24 eps)Apr 2011 - Sep 2011…	24
2	Bleach: Sennen Kessen-henTV (13 eps)Oct 2022...	13
3	Gintama°TV (51 eps)Apr 2015 - Mar 2016...	51
4	Shingeki no Kyojin Season 3 Part 2TV (10 eps)…	10

```
Title       object
Episodes     int64
dtype: object
```

✦ KEY INSIGHT

Numeric aggregations like .mean() or .sum() silently fail or misbehave on object-typed 'number-looking' strings — always cast explicitly before running math on a cleaned text column.

PART 7 • TOPIC 5

Boolean Max-Metric Filtering — Highest Score Identification

Comparing an entire column against its own .max() value produces a boolean mask that isolates the single peak-performing record — the same reusable pattern that powers most 'find the best entry' queries in pandas.

› _ PYTHON

```
boolean_maximum_mask = df_anime['Score'] == df_anime['Score'].max()

highest_rated_anime_titles = df_anime[boolean_maximum_mask]['Title']
```

→ OUTPUT

```
0    Fullmetal Alchemist: BrotherhoodTV (64 eps)Apr...
Name: Title, dtype: object
```

PART 08

Advanced Data Auditing, Filtering & Sorting

Structural Diagnostics, Ranking, Distributions & Regional Masking

9 Core Topics · Fully Worked Examples & Outputs

PART 8 • TOPIC 1

Structural Boundary Diagnostics (.shape, .columns)

Before any analysis begins, .shape and .columns give an instant structural fingerprint of an unfamiliar dataset — dimensions first, feature labels second.

❯_ PYTHON

```
df = pd.read_csv("Countries.csv")

df.shape              # (194, 64)
list(df.columns)      # full 64-column feature label array
```

→ OUTPUT

```
Data Grid Shape Vector: (194, 64)

Extracted Structural Table Features Array:
['country', 'country_long', 'currency', 'capital_city', 'region',
 'continent', 'demonym', … … … 'population', 'median_age',
 'political_leader', 'title']  ← 64 total column labels
```

✦ KEY INSIGHT

Running .shape and .columns first, before any transformation, is standard practice on every unfamiliar dataset — it prevents blind assumptions about what fields actually exist.

PART 8 • TOPIC 2

Global Maximum Value Filtering & Attribute Extraction

The same max-mask pattern from Part 7 scales to any numeric column. Chaining .values[0] after a single-column selection pulls out the raw scalar value instead of a full Series wrapper.

❯_ PYTHON

```
boolean_max_pop_mask =   df['population'] = = df['population'].max()
df_max_pop_record = df[boolean_max_pop_mask]

peak_country_name = df[boolean_max_pop_mask]['country'].values[0]
peak_capital_city = df[boolean_max_pop_mask]['capital_city'].values[0]

print(f “Isolated Country: {peak_country_name} | Labeled Capital: {peak_capital_city}”)
```

→ OUTPUT

```
Isolated Country: India | Labeled Capital: New Delhi
```

PART 8 • TOPIC 3

High-Performance Sorting Architectures (.sort_values)

ascending=False flips the sort direction so the highest values rise to the top of the frame; inplace=True commits the new row order permanently rather than returning a fresh copy.

❯_ PYTHON

```
df.sort_values(by='democracy_score', ascending=False, inplace=True)

top_democracy_entities = df['country'].head(5)
```

→ OUTPUT

```
127         Norway
74          Iceland
164        Sweden
122    New Zealand
46         Denmark
Name: country, dtype: object
```

PART 8 • TOPIC 4

Categorical Value Distribution Audits (.value_counts)

.value_counts() instantly summarizes every unique category and its frequency in one call; chaining .count() afterward tells you how many distinct categories exist in total.

❯_ PYTHON

```
regional_frequency_counts = df['region'].value_counts()

total_distinct_regions = df['region'].value_counts().count()
```

→ OUTPUT

```
region
Western Asia        17
Eastern Africa      17
Western Africa      16
Southern Europe     15
Caribbean           13
…
…
…
Northern America     2
Name: count, dtype: int64

Total Isolated Regional Groups: 22
```

PART 8 • TOPIC 5

Regional Value Lookup & Single-Condition Masking

A specific category's frequency can be pulled directly by label from a .value_counts() result, and the same label re-used to build a standard equality mask that isolates every matching row for that category.

❯_ PYTHON

```
eastern_europe_count = df['region'].value_counts()['Eastern Europe']

boolean_eastern_europe_mask = df['region'] == "Eastern Europe"
df_eastern_europe = df[boolean_eastern_europe_mask]
eastern_europe_countries = df[boolean_eastern_europe_mask]['country']

print(eastern_europe_countries)
```

→ OUTPUT

```
Total Frequency Mapping to 'Eastern Europe': 10

43     Czech Republic
151    Slovak Republic
24          Bulgaria
136           Poland
73           Hungary
139          Romania
111          Moldova
181          Ukraine
14           Belarus
140           Russia
Name: country, dtype: object
```

PART 8 • TOPIC 6

Positional Rank Slicing (.nlargest + .iloc)

Directly indexing for 'the 2nd highest value' by row position is fragile if the frame isn't pre-sorted. .nlargest(n) followed by .iloc[n-1] reliably isolates an exact rank threshold regardless of row order.

❯_ PYTHON

```
target_second_highest_value = df['population'].nlargest(2).iloc[1]

boolean_second_pop_mask = df['population'] == target_second_highest_value
target_leader_identity = df[boolean_second_pop_mask]['political_leader'].values[0]          # Xi Jinping
```

✦ KEY INSIGHT

This two-step 'threshold value → boolean mask' pattern is more robust than direct row-position indexing, since it doesn't assume any particular sort order was already applied.

PART 8 • TOPIC 7

Missing Vector Profiling (.isna Chains)

Chaining .isna() with a following boolean filter and .count() reports exactly how many rows are missing a specific attribute.

› _ PYTHON

```
df['political_leader'].isna().head(5)

total_unverified_leaders_count = df[df['political_leader'].isna()]['country'].count()
```

→ OUTPUT

```
127    False
74     False
164    False
122    False
46     False
Name: political_leader, dtype: bool

Total Rows with Unverified Political Leaders: 7
```

PART 8 • TOPIC 8

Custom String Token Scanning Pipelines

A global tracking counter combined with .apply() lets a custom function scan every row's text for a target keyword, standardizing to lowercase first to avoid case-sensitivity misses.

› _ PYTHON

```
global_matching_keyword_counter = 0

def scan_text_token_patterns(target_input_text):
    global  global_matching_keyword_counter
    if 'republic' in str(target_input_text).lower():
        global_matching_keyword_counter += 1
    return target_input_text

df['country_long'].apply(scan_text_token_patterns)

print(global_matching_keyword_counter)
```

→ OUTPUT

```
Total Country Titles Hosting the Token 'republic': 125
```

✦ KEY INSIGHT

Using a global counter inside .apply() is a simple way to tally pattern matches, but for large-scale pipelines a vectorized .str.contains() call is typically faster.

PART 8 • TOPIC 9

Dual-Conditional Regional Max Filters

Complex 'peak record within a segment' queries are solved in two clean stages: first isolate the regional sub-DataFrame, then apply a second max mask scoped entirely within that smaller subset.

❯_ PYTHON

```
# A. Isolate a targeted geographic segment first
boolean_africa_continent_mask = df['continent'] = = 'Africa'
df_african_continent_segment = df[boolean_africa_continent_mask]

# B. Evaluate the maximum attribute strictly within that filtered subset
boolean_african_peak_pop_mask = (
    df_african_continent_segment['population']
    = = df_african_continent_segment['population'].max()
)

# C. Extract the final targeted row and text attribute
peak_african_record = df_african_continent_segment[boolean_african_peak_pop_mask]
peak_african_country_name = (
    df_african_continent_segment[boolean_african_peak_pop_mask]['country'].values[0]
)

print("\nHighest Populated African Entity Row Summary View:\n", peak_african_record)
print(f"\nFinal Isolated Country Target Value: {peak_african_country_name}")
```

→ OUTPUT

```
Highest Populated African Entity Row Summary View:
  country              country_long       currency  ...  political_leader      title
125 Nigeria  Federal Republic of Nigeria  Nigerian naira  ...  Muhammadu Buhari  President

[54 African rows total → 1 row matched]

Final Isolated Country Target Value: Nigeria
```

✦ KEY INSIGHT

Scoping the second .max() mask to the already-filtered sub-DataFrame is essential — comparing against the GLOBAL max would incorrectly return zero matches within the regional segment.